

# Non-Negative AdderNet: Algorithm-Hardware Co-design for Lightweight Defense of Adversarial Bit-Flip Attacks

Yunxiang Zhang\*, Sabbir Ahmed\*, Abeer Matar A Almalky\*, Adnan Siraj Rakin, and Wenfeng Zhao  
Binghamton University, Binghamton, New York, USA

\*These authors contributed equally to this work.  
arakin@binghamton.edu, wzhaow@binghamton.edu

**Abstract**—AdderNet emerges with superior hardware efficiencies compared to Convolutional Neural Networks (CNNs) by replacing the multiplicative operations with energy-efficient additive alternatives. Nevertheless, AdderNet’s security exploration remains underexplored, especially against emerging adversarial weight perturbation attacks, such as the Bit-Flip Attack (BFA), which injects memory faults into weight bits. In this paper, we investigate AdderNet’s vulnerabilities to BFA for the first time and discover that it is more susceptible to BFA than its CNN counterpart. To utilize the hardware efficiency of AdderNet and at the same time improve resiliency against BFA, we propose a novel Non-Negative AdderNet (NNAN) model. Proposed (NNAN) consists of two key components: i) it incorporates a non-positive weight encoding technique, and ii) a novel accelerator design with an integrated non-positive decoder (NPD) to achieve hardware-level lightweight and runtime detection and correction of BFA. To further test the resilience of our proposed defense, we perform both a baseline BFA and an advanced BFA (BFA+), where the attacker has sufficient knowledge about the proposed defense. Our proposed NNAN makes BFA obsolete and requires many more attack cycles to be compromised against BFA+, successfully enabling the design of both hardware-efficient and secure AdderNet against BFA.

## I. INTRODUCTION

Convolutional Neural Networks (CNNs) have become the leading architecture for feature extraction in visual data analysis [1]. With multi-layered deep structure and convolutional operations, CNNs excel at capturing spatial hierarchies, making them indispensable for tasks such as object detection [2], image classification [1], and semantic segmentation [3]. However, CNNs necessitate a high volume of multiplications, which not only increases computational complexity but also significantly raises deployment costs such as energy consumption, especially in resource-scarce settings.

Optimization techniques such as quantization [4]–[7] and pruning [5], [8] have been proposed to address the computational burden associated with multiplications. These techniques aim to reduce model size (i.e., memory) and computational load (i.e., energy cost) by lowering the bit precision and removing redundant parameters. While pruning and quantization are highly effective for model compression, innovative DNN architecture design can further support/augment their energy and memory gains. As a result, a tangible line of research has introduced AdderNet [9] as a promising hardware-

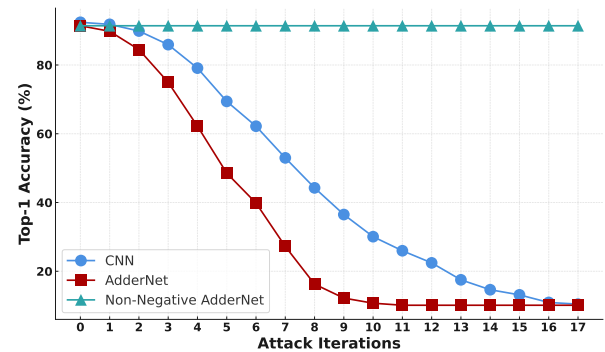


Fig. 1. The evaluation of Bit Flip Attack (BFA) on CNN, AdderNet, and Non-Negative AdderNet (ResNet20 on CIFAR10 dataset). The results demonstrate that AdderNet is more susceptible to BFA than CNN. Our proposed NNAN effectively defends BFA and preserves accuracy by detecting and correcting bit-flips.

friendly alternative to CNNs. AdderNet offers an efficient sum-of-absolute-difference (SAD) feature extraction kernel that replaces costly multiplications with additions, which improves hardware efficiency while simultaneously achieving comparable model accuracy compared with the CNN counterpart.

Previous studies have demonstrated that CNNs are vulnerable to the prominent adversarial weight perturbation attack, i.e., the Bit-Flip Attacks (BFA) [10]–[18], where the attacker injects targeted memory faults into DNN weight bits through rowhammer [19]. These works concluded that a minimal number ( $< 0.00001\%$ ) of bits flipped in the model weights can substantially affect CNN model performance. Although the security of CNNs has been widely examined, the security profile of its efficient counterpart AdderNet against BFA remains largely under-explored in prior research. While AdderNet indeed possesses great potential in achieving improved hardware efficiency, *does such benefit come at the cost of compromised model resiliency against weight perturbation attack such as BFA?* If this is the case, *how can we achieve AdderNet design that is both efficient and secure against BFA?*

In this work, we investigate for the first time the vulnerabilities of AdderNet to the prevalent adversarial weight perturbation attack, namely Bit-Flip Attack (BFA), providing evidence of its susceptibility. Our initial analysis of performing BFA on AdderNet indicates that the model performance is

severely compromised by flipping an even smaller number ( $<10$ ) of bits compared to CNN, as shown in Fig. 1. *Hence, we conclude that AdderNet leveraging additive operation is more vulnerable against BFA. Such observation provides the foundation for our research to further explore both efficient and secure AdderNet design against weight manipulations.*

In addition, we propose a novel secure Non-Negative AdderNet (NNAN) model exploiting a mathematically equivalent transformation and Non-Positive Encoding (NPE) technique over model weights. NNAN leverages a similar additive principle as AdderNet for feature extraction but with enhanced robustness against BFA, thanks to the non-negative kernel operation and the NPE scheme. NNAN and NPE encoding can be efficiently and seamlessly integrated into existing accelerator architecture, serving the purpose of lightweight and runtime detection and correction of bit-flips against advanced attackers with the knowledge of the defense. The contributions of this paper are summarized as follows:

- We empirically examine, for the first time, the AdderNet vulnerability against BFA. *Our main observation is that, AdderNet is indeed vulnerable against BFA and shows worse resistance compared to CNN (refer to Fig. 1).*
- We propose NNAN coupled with a Non-Positive Encoding (NPE) scheme that achieves lightweight and runtime bit-flip detection and correction to thwart BFA (refer to Fig. 1).
- We demonstrate a novel NNAN accelerator design with an integrated lightweight non-positive decoder (NPD) to achieve hardware-level runtime BFA detection and correction (BFADC). Experimental results on AMD/Xilinx Kria-260 suggest only 5.5% LUT overhead due to the added NPD without incurring any latency or training overhead.
- First proposed NNAN completely defends BFA. For robust evaluation, we consider an advanced attacker with knowledge of the detection scheme. This knowledgeable attacker can redesign the BFA algorithm to bypass the defense, which we term it as BFA+. *Our evaluation shows that even against this strong BFA+, the proposed NNAN shows improved attack resistance compared to CNN counterpart against BFA only.*

This paper is organized as follows. Section II introduces the background of AdderNet and Bit-Flip Attack. Section III demonstrates the observations on BFA to vanilla AdderNet. Section IV details our proposed Non-Negative AdderNet (NNAN) and Non-Positive-Encoding Technique for BFA Detection and Correction (BFADC). Section V presents the hardware architecture of NNAN by employing BFADC. Section VI presents the experimental results. Section VII concludes the paper.

## II. BACKGROUND

### A. Preliminaries on AdderNet

In a convolutional layer of a CNN model, let  $W \in \mathbb{R}^{k \times k \times c_i \times c_o}$  denotes a 4D weight tensor, where  $k$  is the kernel

size,  $c_i$  and  $c_o$  denote the number of input and output channels, respectively. For a given input feature map  $X \in \mathbb{R}^{h \times w \times c_i}$ , with  $h$  and  $w$  representing the height and width of the feature map, the cross-correlation between the input and weight tensor produces an output feature map  $Y$ :

$$Y(m, n, c_o) = \sum_{i=1}^k \sum_{j=1}^k \sum_{l=1}^{c_i} X(m+i, n+j, l) \times W(i, j, l, c_i), \quad (1)$$

where  $Y$  captures local features through the convolutional filters serving as a similarity measure between the input and the weights. In an AdderNet model [9], the convolution operations are substituted with sum-of-absolute-difference operations, i.e., utilizing the  $\ell_1$ -norm distance metric between the input feature map and the filter kernels. Thus, the output feature  $Y$  can be rewritten as:

$$Y(m, n, c_o) = - \sum_{i=1}^k \sum_{j=1}^k \sum_{l=1}^{c_i} |X(m+i, n+j, l) - W(i, j, l, c_i)|. \quad (2)$$

In this way, AdderNet results in significant reduction in hardware resource usage and energy consumption. Notably, AdderNet serves as an efficient drop-in replacement for CNN architectures, allowing most intermediate convolutional layers to be swapped with adder layers.

### B. AdderNet Model Quantization

Despite of the theoretical benefits of efficient underlying additive operations, AdderNet still faces distinct quantization challenges, while weight quantization techniques for CNNs cannot be generalized to AdderNet. Unlike CNNs, where weight and ReLU activation quantization can be performed independently, in AdderNet, the quantization step size ( $\Delta$ ) for ReLU activations and subsequent layer weights must be the same to ensure accurate SAD operations. Additionally, the weights in AdderNet exhibit a zero mean Laplace distribution [9] but with a significantly larger variance compared to ReLU activations. Hence, a larger  $\Delta$  can indeed facilitate lower weight quantization bitwidth, it adversely affects model accuracy due to diminishing ReLU activations during quantization.

As a result, baseline AdderNet weights can only be quantized down to 10-bit [9], which offsets the anticipated hardware advantages of SAD kernels. Subsequent low-bitwidth quantization schemes have achieved INT8 quantization using weight standardization (WS) technique [20], as the weight dynamic range is reduced by approximately four times. However, WS leads to model accuracy drop (around 1%) compared to the baseline AdderNet.

Alternatively, an activation-oriented quantization (AOQ) scheme was introduced in [21], which achieves INT6 quantization with a weight clipping technique. For each layer, the outputs from batch normalization (BN) undergo standard scalar quantization, resulting in quantized  $\text{BN}(\cdot)$  and  $(q-1)$ -bit non-negative ReLU outputs:

$$\text{BN}(\cdot) \in [-2^{q-1}, 2^{q-1} - 1], \quad (3)$$

$$\text{ReLU}(\cdot) \in [0, 2^{q-1} - 1], \quad (4)$$

The quantization step size  $\Delta$  will then be used to quantize the weights  $W$ , yielding a bitwidth exceeding  $q$ . Hence, a weight clipping process is performed to achieve  $q$ -bit quantization:

$$\mathbf{W}_{clip}(w) \in [-2^{q-1}, 2^{q-1} - 1] \quad (5)$$

The difference between the original weights  $W$  and the clipped weights  $W_{clip}$  is termed as the weight bias:

$$\mathbf{W}_{bias} = -|\mathbf{W} - \mathbf{W}_{clip}|, \quad (6)$$

while the mathematical properties of SAD operations ensures:

$$-\sum |\mathbf{X} - \mathbf{W}| \equiv -\sum |\mathbf{X} - \mathbf{W}_{clip}| + \sum \mathbf{W}_{bias}, \quad (7)$$

where  $\sum |\mathbf{X} - \mathbf{W}_{clip}|$  represents  $q$ -bit clipped SAD operations, and  $\sum \mathbf{W}_{bias}$  is a constant after model training and can be pre-computed without incurring extra hardware overheads. Furthermore, as this scheme clips the weights to a specific bitwidth, it may also offer some advantages against Bit-Flip based adversarial weight attacks. Compared to non-clipped weights, the impact of accuracy degradation from such attacks may be reduced.

### C. Bit-Flip based Adversarial Weight Attack

Bit-Flip Attack (BFA) [10] is an adversarial attack designed to manipulate neural network weights by selectively flipping the most vulnerable bits in their binary representation. The attacker iteratively performs two essential steps: (i) an inter-layer search and (ii) an intra-layer search. These two steps are conducted to identify vulnerable weight bits that can be flipped to maximize stealthiness of the attack.

In the context of a quantized deep neural network (DNN), the weight matrix is represented using binary representations,  $\{B_l\}_{l=1}^L$ , where  $l$  denotes layers index from 1 to  $L$ . During the intra-layer search, the gradient impact of each weight bit is evaluated to identify the bit with the highest gradient as a candidate for vulnerability:

$$\arg \max_{B_l} \|\nabla_{B_l} \mathcal{L}\|, \quad (8)$$

where  $\mathcal{L}$  is the inference loss. Afterward, the inter-layer search compares these candidate bits to find modifications that will lead to the most significant degradation in the model performance. Given an input sample  $x$  and its corresponding label  $t$ , BFA algorithm seeks to maximize the following loss function:

$$\max_{\{\hat{B}_l^i\}} \mathcal{L}(f(x; \{\hat{B}_l^i\}_{l=1}^L), \tilde{t}), \quad (9)$$

where  $\hat{B}_l^i$  represents the quantized bit tensor of  $l$ -th layer perturbed by BFA in iteration  $i$ , while  $f(\cdot)$  is the outputs of DNN.  $\tilde{t}$  is clean model output as soft-labels. The effectiveness of the attack is evaluated by calculating the Hamming distance:

$$\sum D(\hat{B}_l^i, B_l), \quad (10)$$

which quantifies the number of bit flips between post-attack ( $\{\hat{B}_l^i\}_{l=1}^L$ ) and pre-attack ( $\{B_l\}_{l=1}^L$ ) model parameters. The primary objective of BFA is to degrade the performance of DNN with minimal number of bit flips.

TABLE I  
STANDARD THREAT MODEL FOR BFA [10], [14] AND OUR REDESIGNED BFA+ TO BYPASS PROPOSED DETECTION DEFENSE.

| Information Attackers Poses           | BFA | BFA+ |
|---------------------------------------|-----|------|
| Model architecture and parameters     | ✓   | ✓    |
| Small batch (e.g., 128) of test data  | ✓   | ✓    |
| Address of parameter cached in DRAM   | ✓   | ✓    |
| Model training data and configuration | ×   | ×    |
| Memory read & write permission        | ×   | ×    |
| Detailed knowledge about defense      | ×   | ✓    |

### D. Threat Model

In this paper, we assume a similar conventional white-box threat model, which is the standard for BFA in prior research [10], [22]. As summarized in Table I, this threat model assumes that the attacker has detailed knowledge of the DNN internal structure, such as layers count and dimensions. In addition, the attacker is assumed to know the DNN parameters, their values, and bit-level representation for inference purposes. The aforementioned assumption is realistic because of the recent advancements in side-channel data leakage and DNN model reverse-engineering techniques [23], which can uncover the DNN configuration during inference. Although the attacker has white-box access, attacker capabilities are restricted due to lacking access to the training configuration and data. Attackers have only a sample batch of test data available to initiate the attack. Additionally, the attacker is co-located with the victim, which allows an attacker to run unprivileged user-space processes. Furthermore, the attacker has full knowledge of the DRAM addressing scheme.

In this study, we examine two types of white-box attack settings: (i) *Conventional BFA*, known as *BFA* where the attacker is unaware of the presence of any defense mechanism. (ii) *BFA+*, which assumes the attacker is aware of a defense being placed and knows the details of the proposed encoding mechanism. Table I summarizes the corresponding assumptions for both attacks. Finally, the attacker can leverage any popular side-channel attacks fault injection mechanism such as RowHammer [13], [14], [19] or RowPress [24] to successfully realize precise targeted bit-flips in the DRAM.

## III. MOTIVATING OBSERVATIONS

Before introducing our proposed approach  $\overleftarrow{\text{NNAN}}$ , we first present the problem our work aims to address: the vulnerability of AdderNet under Bit-Flip Attacks (BFA). In this section, we provide the first empirical demonstration of the BFA threat to AdderNet and a motivating analysis towards its protective measure. We perform BFA against both AdderNet and CNN where the following two observation stands out:

- **Observation I:** As shown in Fig. 1, BFA on AdderNet is more effective than CNN, as only a handful amount of bit flips are required to successfully compromise the model.
- **Observation II:** Table II summarizes that BFA almost always attacks the most significant bits of both CNN and AdderNet.

TABLE II  
BIT FLIP DISTRIBUTION IN BOTH ADDERNET AND CNN (RESNET20 ON CIFAR10 DATASET): ALL ATTACKS ARE MADE TO MSBs.

| Model    | MSB   | 2 <sup>nd</sup> MSB | 3 <sup>rd</sup> MSB | 4 <sup>th</sup> MSB |
|----------|-------|---------------------|---------------------|---------------------|
| CNN      | 100 % | 0 %                 | 0 %                 | 0 %                 |
| AdderNet | 100 % | 0 %                 | 0 %                 | 0 %                 |

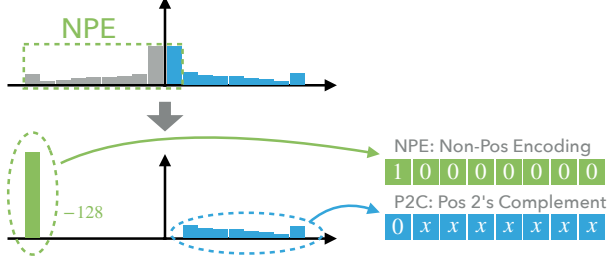


Fig. 2. Proposed Non-Positive Encoding Scheme.

Based on the aforementioned observations, we conclude that: despite of the promising hardware-level benefits, vanilla AdderNet architecture and its weight storage are more vulnerable to BFA and require innovative defense solutions. Protecting the MSBs is a plausible direction to develop secure AdderNet.

#### IV. PROPOSED METHOD

In this section, we present  $\overleftarrow{\text{NNAN}}$ , a novel Non-Negative AdderNet paradigm with a Non-Positive Encoding technique (NPE) to thwart BFA. Through NPE scheme,  $\overleftarrow{\text{NNAN}}$  possesses the capability to achieve online detection and correction of BFA targeting the sign bits (i.e., MSBs) of the model weights.

##### A. NN-AdderNet

In this subsection, we first detail the mathematical foundation of Non-Negative AdderNet.

For a ReLU-activation based AdderNet model, the computation to obtain one output feature pixel is the SAD operations between the input features and the weight kernels across all input channels. After model training, both positive and negative weights are determined, leaving only the feature maps across different layers changing with respect to different inputs. Hence, below transformation to the SAD kernel function is mathematically sound:

$$\begin{aligned}
 \sum |x - w| &= \sum |x_p - w_p| + \sum |x_n - w_n| \\
 &= \sum |x_p - w_p| + \sum |x_n - 0| + \sum |w_n| \quad (11) \\
 &= \sum |x - \text{ReLU}(w)| + \sum |w_n|,
 \end{aligned}$$

where  $w_p$  and  $w_n$  represent the positive and negative weights, while  $x_p$  and  $x_n$  are the corresponding activations, respectively.

Eq. (11) holds valid with the assumption of neural network models with ReLU activations, which is the widely adopted activation function used in popular DNN models. Apparently, Eq. (11) suggests that the basic SAD computation with signed weights is mathematically equivalent to the unsigned SAD computation with “ReLU-ed weights” plus the sum of the

TABLE III  
BFA DETECTION AND CORRECTION OF NPE.

|     | Encoded   | BFA       | Detected | Recovered |
|-----|-----------|-----------|----------|-----------|
| NPE | 1000_0000 | 0000_0000 | ✓        | 0000_0000 |
| P2C | 0xxx_xxxx | 1xxx_xxxx | ✓        | 0xxx_xxxx |

absolute of all negative weights  $\sum |w_n|$ . Note that  $\sum |w_n|$  can be pre-computed after model training and ultimately handled by fuse-bias removal technique [21], which will not incur additional computation overheads during model inference. Hence, Eq. (11) demonstrates the feasibility of NN-AdderNet, where negative weights contribution to the AdderNet model computation will be all fused to a *feature bias* that can be removed by Batch Normalization, leaving only positive weights matter to the computation. Moreover, Eq. (11) holds valid for both floating-point and quantized operations, which can be seamlessly applied with the activation-oriented quantization scheme described in Section II-B.

Intuitively, an INT- $n$ -bit AdderNet is equivalent to Unsigned- $(n-1)$ -bit NN-AdderNet via sign bit saving. Instead, we demonstrate the utility of the sign bit savings in NN-AdderNet for ultra lightweight, online detection and correction of BFA flipped bits through a novel non-positive weight encoding strategy, as will be detailed in the next subsection.

##### B. Non-Positive Encoding for Secure NN-AdderNet

The non-negative weights in NN-AdderNet can be obtained via ReLU operations on the signed weights, where all the negative weights are clipped to zero. Nevertheless, the unsigned binary encoding of the non-negative weights still needs a dedicated encoding scheme to enable BFA detection and correction capabilities.

To this end, we propose a provable-secure non-positive weight encoding (NPE) scheme, in which we leverage the original sign bits to achieve secure NN-AdderNet. Fig. 2 shows a conceptual illustration on the proposed NPE scheme. Using 8-bit weight as an example, we encode all the non-positive weights in NN-AdderNet as 1000\_0000, i.e., ‘-128’ in 2’s complement format. All positive weights still follow the basic positive 2’s complement encoding (P2C) with a format of 0xxx\_xxxx, where there is at least one bit of ‘1’ exists in the non-MSB part.

The detailed BFA detection and correction mechanism is demonstrated in Table. III. If BFA is applied to the encoded weights, the sign bits are flipped and lead to 0000\_0000 for NPE and 1xxx\_xxxx for P2C, respectively. According to the NPE encoding strategy outlined in previous paragraph, we know that both ‘0’ and negative values except ‘-128’ do not exist in our encoding scheme. Hence, BFA can be detected in both cases. We will only need to perform correction and deliver the non-negative weights 0000\_0000 and 0xxx\_xxxx to the accelerator. On the contrary, if the encoded weights are not BFA targeted, then a similar decoding scheme will be applied, passing non-negative weights to the accelerator.

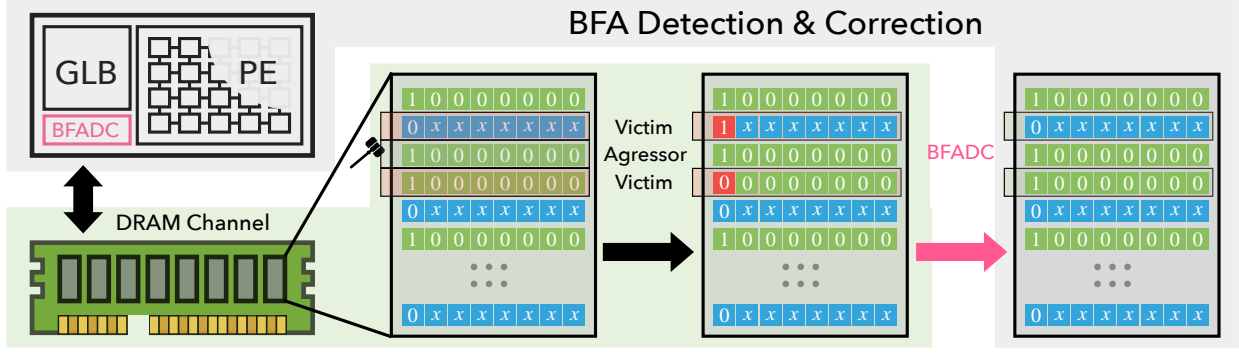


Fig. 3. Secure NN-AdderNet Accelerator Architecture  $\overleftarrow{\text{NNAN}}$  with lightweight online BFA Detection and Correction capability.

In sum, when leveraging the sign bit in NN-AdderNet with non-positive encoding strategy, the redundant nature allows successful BFA detection and correction as the attacked weights do not exist in our encoding scheme. Hence, BFA threat to NN-AdderNet with NPE is completely eliminated! Such capability also paves the way towards lightweight online detection and correction of BFA at hardware level.

## V. HARDWARE ARCHITECTURE FOR $\overleftarrow{\text{NNAN}}$ ACCELERATOR

In this section, we present a case study on the secure NN-AdderNet accelerator architecture for  $\overleftarrow{\text{NNAN}}$ , as illustrated in Fig. 3. In this setting, DNN model weights are stored in off-chip DRAM, which is known to be vulnerable to rowhammer-based BFA [19]. Frequent aggressor row access will cause bit flips in the victim rows that can be exploited by the attacker. After model training, AdderNet will undergo model quantization and the proposed non-positive weight encoding process. During deployment, the NPE encoded weights are stored in DRAM and can be BFA targeted.

### Algorithm 1 HLS Pseudo Code for BFADC

```

Input:    Encoded 8-bit Weight (EW) with size of  $T_{out} \times T_{in} \times K_R \times K_C$ 
Output:   Decoded 8-bit Weight (DW) with size of  $T_{out} \times T_{in} \times K_R \times K_C$ 
for ( $k_R = 1 : K_R, k_C = 1 : K_C$ ) do
  for  $t_R = 1 : T_R, t_C = 1 : T_C$  do
    for  $t_{out} = 1 : T_{out}$  do
      for  $t_{in} = 1 : T_{in}$  do
        if  $EW == -128$  or  $EW == 0$  then
           $DW = 0$ 
        else if  $EW < 0$  then
           $DW = EW + 128$ 
        else
           $DW = EW$ 
        end if
      end for
    end for
  end for
end for

```

With the proposed  $\overleftarrow{\text{NNAN}}$ , online BFA Detection and Correction (BFADC) becomes feasible and practical, thanks to the redundancy created by both proposed schemes in NN-AdderNet. When encoded weights are loaded from DRAM to

the accelerator during inference, a BFADC block (illustrated in Fig. 3) will be employed before the encoded weights are loaded to the global buffer (GLB). Basically, the BFADC block serves two purposes: (i) when the encoded weights are not targeted by BFA, a normal decoding will be performed; and (ii) when BFA is performed on the encoded weights, the BFADC block can detect the BFA and ultimately correct the affected weights into correct ones before writing to the GLB. Algorithm 1 shows the HLS pseudo code for the lightweight BFADC block. Note that the BFADC logic poses negligible hardware overheads to the NN-AdderNet accelerator, making it ideal for online lightweight BFA detection and correction. Detailed experimental results will be shown in Section VI.

## VI. EXPERIMENTAL RESULTS

### A. Experimental Setup

1) *Datasets and Network Architectures:* We evaluate our proposed  $\overleftarrow{\text{NNAN}}$  on three widely recognized visual datasets: CIFAR-10 [25], CIFAR-100 [25], and ImageNet [26], which are extensively utilized as standard benchmarks for BFA evaluation [10], [14]. The ResNet-20, VGG-Small, ResNet-32, and SqueezeNet models were trained in FP32 precision on a GeForce RTX 4070 Super GPU using CUDA 10.2 and PyTorch 2.0 under Ubuntu 20.04 LTS. For ResNet-20 and VGG-Small, the initial learning rate was set to 0.05, while ResNet-32 and SqueezeNet used 0.1. All models were trained with a batch size of 256 using the SGD optimizer with a momentum of 0.9 and a weight decay coefficient of  $5 \times 10^{-4}$ , along with an adaptive learning schedule employing cosine learning rate decay [27]. For ResNet-50, a pre-trained model from [9] was adopted<sup>1</sup>.

All models were processed with the proposed NPE scheme in the NN-AdderNet setting and quantized to INT8. In general, we store any convolution and FC layers in the AdderNet (typically the first and last few layers) in the on-chip memory with the accelerator. This is standard practice for quantized neural network accelerator design [20] and BFA evaluation [28] (attacking only layers stored off-chip). For fair attack evaluation, the same protocol was maintained for CNN evaluation as well.

<sup>1</sup><https://tinyurl.com/32wfn5ph>



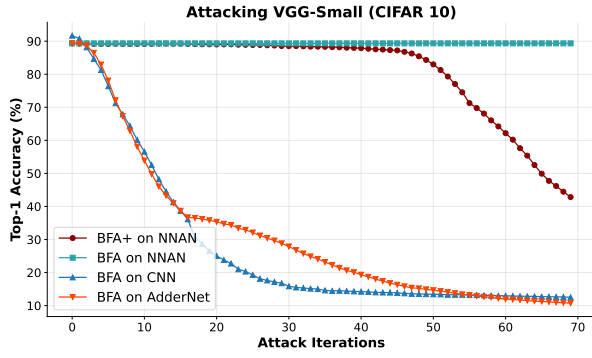


Fig. 4. Comparison of BFA across three network variants of VGG-Small. Our proposed  $\overline{\text{NNAN}}$  demonstrates a strong defense against BFA through bit-flip detection and correction. Even against BFA+,  $\overline{\text{NNAN}}$  still manages to resist the attack's impact to only below 40% upto 80 iterations.

2) *Attack Settings*: In this study, we examine two different BFA attacks based on the white-box threat model settings discussed before. *The first setting is BFA* [10], [11], [14], [22], where the attacker is unaware of any defense mechanism in place and attempts to exploit the network without any prior knowledge of protective measures. Hence to evaluate against BFA, we follow the same attack algorithm developed in prior works [10], [11], [14], [22] for BFA. In our evaluation, we report the number of bit-flips (i.e., attack rounds) required to deplete the model performance to random guess (e.g., 10 % for a 10-class classification) or pre-defined amount of maximum attack iteration (e.g., 80), whichever comes first.

*The second setting is BFA+*, in which the attacker is assumed to be aware of the presence of a defense and can infer certain properties of the encoding scheme. In this scenario, the attacker reverse-engineers the detection scheme by performing repeated BFAs and runs a statistical analysis of the failure when targeting MSB and second MSB weights (non-positive weights). As a result, the attacker in BFA+ can improve the attack efficacy even further by ensuring the bit searching algorithm intentionally avoids attacking MSB and second MSB weights (non-positive weights). We achieve this by setting the gradient of all MSB and second MSB weights (non-positive weights) to zero during the raking step of the BFA algorithm in Eq. (8).

3) *Hardware Evaluation Setup*: Xilinx Kria KV-260 Vision AI starter kit was used as the hardware implementation platform, whereas the programming logic (PL) part has 1,248 DSP, 117-kLUT, and 144 BRAM36. We adopt the spatial accelerator architecture and state-of-the-art INT8 SAD DSP packing strategy [21] for implementing NN-AdderNet accelerators. Using this scheme, each DSP48E2 slice with a few LUTs on the KV-260 board can be configured to pack Octo-INT8 SAD operations. The working frequency of the accelerators running on KV-260 was set to 200 MHz. The NN-AdderNet accelerator designs were implemented in HLS C using Xilinx Vitis HLS and Vivado 2022.1. DSP-LUT co-packing was achieved by manually instantiating a DSP slice in the HLS-generated Verilog RTL design. The final design

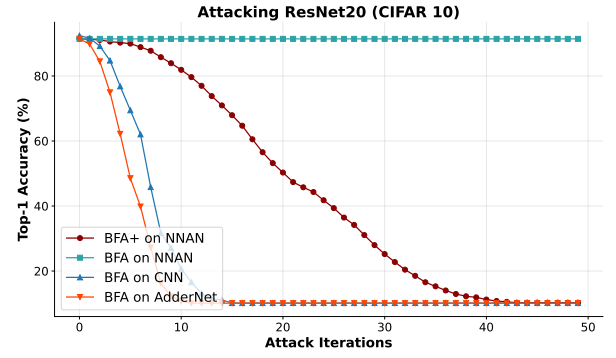


Fig. 5. Comparison of BFA across three network variants of ResNet20. Our proposed  $\overline{\text{NNAN}}$  demonstrates a strong defense against BFA through bit-flip detection and correction.  $\overline{\text{NNAN}}$  still maintains better resilience than CNN against BFA+.

will be packaged as a Vivado IP to generate the bitstream and .hwh file for PYNQ framework.

## B. Experimental Results

### 1) Evaluation of $\overline{\text{NNAN}}$ on Different DNNs and Datasets:

a) *CIFAR-10 Evaluation.*: We evaluate the conventional BFA scenario on various architectures for CIFAR-10. First, for ResNet-20, as shown in Fig. 5, AdderNet is more susceptible to accuracy degradation against BFA than CNN. Both CNN and AdderNet accuracy drops to a random guess level within only 10 attack iterations. In contrast, our proposed  $\overline{\text{NNAN}}$ , with the support of NPE encoding, can protect NN-AdderNet against BFA (green curve), rendering BFA obsolete. Next, we evaluate our robust AdderNet against BFA+; our proposed defense still exhibits improved resilience to BFA. It requires almost  $2\times$  more attack iterations compared to CNN against BFA to deplete the model's intelligence at the random guess level, significantly improving model robustness against bit flips.

Our conclusion holds for different architectural topologies, such as VGG-small, as shown in Fig. 4. Again, AdderNet remains equally vulnerable against BFA as CNN. Both Base-line AdderNet and CNN accuracy drop to 10 % after 70 attack iterations. However, with the protection of  $\overline{\text{NNAN}}$ , we could successfully protect against BFA through 100 % bit-flip detection and correction. A general observation is that the VGG architecture is inherently more resilient to bit flips. This conclusion holds true for BFA+, which fails to degrade model performance beyond 40 % for 80 attack iterations. This observation highlights that larger models inherently pose some form of resilience to adversarial noise, which is also validated by prior works [10], [29], and our proposed defense consistently shows better resilience in those models as well.

b) *CIFAR-100 and ImageNet Evaluation.*: Additionally, we evaluated our approach on two different datasets: CIFAR-100 and ImageNet. As shown in Table IV, CIFAR-100 was evaluated using ResNet-32 and SqueezeNet, while ImageNet was evaluated using ResNet-50. In CIFAR-100, for ResNet32 and SqueezeNet, AdderNet suffers from slight accuracy degradation compared to CNN. Nevertheless, our attack evaluation

TABLE IV

EVALUATION OF BFA ON ADDERNET AND  $\overline{\text{NNAN}}$  (BFA) MODELS TRAINED ON CIFAR100 AND IMAGENET DATASET AND EVALUATED AGAINST BFA.  $\overline{\text{NNAN}}$ (BFA+) MEANS THE MODEL IS EVALUATED UNDER BFA+ ATTACK AND ITERATIONS SHOWS NUMBER OF BIT FLIPS REQUIRED TO DEGRADE MODEL ACCURACY TO RANDOM GUESS LEVEL. THE RESULTS HIGHLIGHT THAT ADDERNET IS HIGHLY VULNERABLE TO BFA WHEREAS  $\overline{\text{NNAN}}$  COMPLETELY DEFENDS IT. EVEN UNDER STRONGER BFA+ ATTACK  $\overline{\text{NNAN}}$  SHOWS STRONG RESILIENCE REQUIRING  $2\text{--}4\times$  HIGHER ATTACK ITERATIONS.

| Model      | Dataset  | Method                          | Acc.(%) | Iterations          |
|------------|----------|---------------------------------|---------|---------------------|
| ResNet32   | CIFAR100 | AdderNet                        | 67.94   | 9                   |
|            |          | $\overline{\text{NNAN}}$ (BFA)  | 67.94   | <b>Attack Fails</b> |
|            |          | $\overline{\text{NNAN}}$ (BFA+) | 67.94   | 17                  |
| SqueezeNet | CIFAR100 | AdderNet                        | 69.13   | 9                   |
|            |          | $\overline{\text{NNAN}}$ (BFA)  | 69.13   | <b>Attack Fails</b> |
|            |          | $\overline{\text{NNAN}}$ (BFA+) | 69.13   | 22                  |
| ResNet-50  | ImageNet | AdderNet                        | 74.15   | 8                   |
|            |          | $\overline{\text{NNAN}}$ (BFA)  | 74.15   | <b>Attack Fails</b> |
|            |          | $\overline{\text{NNAN}}$ (BFA+) | 74.15   | 33                  |

TABLE V  
ACCELERATOR RESOURCE COSTS WITH OR W/O BFADC.

|                          | LUT   | FF    | DSP | BRAM36 | Latency  |
|--------------------------|-------|-------|-----|--------|----------|
| Baseline                 | 48556 | 22034 | 136 | 48     | 1.113 ms |
| $\overline{\text{NNAN}}$ | 51320 | 22037 | 136 | 48     | 1.113 ms |

follows a similar trajectory across datasets and architectures. Again, AdderNet is extremely vulnerable to BFA, requiring fewer than 10 attack iterations to compromise model performance. Nevertheless, proposed  $\overline{\text{NNAN}}$  successfully defended BFA. In addition, against the adaptive attacker of BFA+, our defense consistently exhibits almost  $2\times$  more resiliency. Similarly, for ImageNet IV, AdderNet exhibits slight accuracy degradation compared to standard CNNs. However, following the trend,  $\overline{\text{NNAN}}$  consistently defends BFA and demonstrates high resilience against BFA+. Proposed defense again makes BFA obsolete and increases the attack iteration by  $8\times$  on ResNet-50 (on ImageNet) evaluated against BFA+ compared to CNN against BFA.

In general, our attack evaluation follows a similar trajectory across different datasets and models. The proposed defense effectively mitigates BFA and improves robustness against BFA+ when compared to both baseline AdderNet and CNN (against BFA).

2) *Hardware Overhead of  $\overline{\text{NNAN}}$* : Table V summarizes the resource costs of baseline AdderNet and secure  $\overline{\text{NNAN}}$  accelerator designs for ResNet-20 model, respectively. The  $\overline{\text{NNAN}}$  accelerator was designed to support unsigned 8-bit (UINT8) model with the corresponding online 8-bit BFADC decoder immediately after the AXI4 interface. For the sake of rigorous comparison, the baseline accelerator was designed to execute an equivalent vanilla INT7 AdderNet model. As can be observed, the proposed  $\overline{\text{NNAN}}$  achieves identical inference performance, DSP and BRAM usage, as well as negligible DFF increase, as compared with the baseline AdderNet accelerator. The implementation overheads of  $\overline{\text{NNAN}}$ , in terms of LUTs utilization, shows a minor 5.5% LUTs overheads.

TABLE VI

EVALUATION OF BFA ON RESNET-20 TRAINED WITH DIFFERENT METHODOLOGIES. ITERATIONS SHOW THE NUMBER OF BIT FLIPS REQUIRED TO DEGRADE MODEL ACCURACY TO RANDOM GUESS LEVEL.

| Training Method                 | Acc.(%)                     | Iterations          |
|---------------------------------|-----------------------------|---------------------|
| AdderNet                        | 91.41                       | 11                  |
| Robust AdderNet [33]            | 89.59 ( $\downarrow 1.82$ ) | 7( $\downarrow 4$ ) |
| $\overline{\text{NNAN}}$ (ours) | 91.41 ( $\downarrow 0$ )    | <b>Attack Fails</b> |

This increase is in part due to the reduced adders from the parallel processing element (UINT8 to INT7, 1.8%), and in part due to the parallel BFADC decoders (Algorithm 1, 3.7%) for online BFA detection and correction. These results successfully demonstrates  $\overline{\text{NNAN}}$  a lightweight solution to defend BFA to the AdderNet model.

### C. Related Works and Comparison

While there are recent works [28], [30]–[32] that attempted to defend against BFA on the CNN counterpart, none of them are uniquely designed for the hardware-efficient AdderNet we are concerned with in this work. Nonetheless, these works are not necessarily competing methods to our defense, as they all can be incorporated into our proposed  $\overline{\text{NNAN}}$  framework. As an orthogonal approach to our defense, their implementation comes at the expense of additional training requirements and model accuracy reduction.

If we take two of the most recent popular BFA defenses, such as Aegis [30] and Neurpots [31], which often come with additional training overhead and accuracy cost. First, both of them need to modify the training framework, including training loss, incorporate architectural changes, and perform training. Whereas our defense is directly applied to an off-the-shelf pre-trained AdderNet w/o incurring any training overhead. In addition, both these defenses, which require additional training, often come with model performance degradation in the range of 0.67 to 1.96%. In contrast, the proposed  $\overline{\text{NNAN}}$  framework, which can be applied to any pre-trained model, does not incur this training overhead nor accuracy degradation, while ensuring 100% detection and correction rate.

In Table VI, we further explore a different line of related work where the goal is to develop robust versions of AdderNet [33]. The design motivation for Robust AdderNet was to defend against input adversarial noise [29]. However, our evaluation of [33] shows that protecting against input noise does not necessarily protect against adversarial weight noise. Rather, Robust AdderNet [33] is more susceptible to weight noise than baseline AdderNet. This further motivates and validates the design of a novel AdderNet architecture designed to protect against adversarial bit-flip attacks.

## VII. CONCLUSION

In this paper, we investigate the vulnerability of an emerging hardware-optimal and energy-efficient DNN model, AdderNet, against Bit-Flip Attacks (BFA) for the first time. Our initial discovery suggests that AdderNet is more vulnerable to BFA. Inspired by this, we introduce a novel secure AdderNet variant,

Non-Negative AdderNet (NNAN), which employs a non-positive weight encoding technique to safeguard AdderNet against BFA. We perform extensive experiments on different model architectures and three different datasets to evaluate the performance of the proposed method in detecting and eliminating BFA. Our results show that the proposed method (NNAN) demonstrates efficient online detection and correction of BFA and improves resilience against advanced versions of BFA+. We hope this research will expand the possibilities of adopting AdderNet in security-sensitive applications.

#### ACKNOWLEDGMENT

This work was in part supported in part by the Binghamton University Transdisciplinary Areas of Excellence (TAE) Data Science grant, and in part by the funding from Integrated Electronics Engineering Center (IEEC) at Binghamton University, NYS Division of Science, Technology and Innovation.

#### REFERENCES

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems*, F. Pereira, C. Burges, L. Bottou, and K. Weinberger, Eds., vol. 25. Curran Associates, Inc., 2012.
- [2] S. Ahmed, U. Kamal, and M. K. Hasan, "Dfr-td: A deep learning based framework for robust traffic sign detection under challenging weather conditions," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 6, pp. 5150–5162, 2021.
- [3] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 3431–3440.
- [4] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 2704–2713.
- [5] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," *arXiv preprint arXiv:1510.00149*, 2015.
- [6] M. Courbariaux, I. Hubara, D. Soudry, R. El-Yaniv, and Y. Bengio, "Binarized neural networks: Training deep neural networks with weights and activations constrained to+ 1 or-1," *arXiv preprint arXiv:1602.02830*, 2016.
- [7] M. Courbariaux, Y. Bengio, and J.-P. David, "Binaryconnect: Training deep neural networks with binary weights during propagations," *Advances in neural information processing systems*, vol. 28, 2015.
- [8] S. Han, J. Pool, J. Tran, and W. Dally, "Learning both weights and connections for efficient neural network," *Advances in neural information processing systems*, vol. 28, 2015.
- [9] H. Chen, Y. Wang, C. Xu, B. Shi, C. Xu, Q. Tian, and C. Xu, "Addernet: Do we really need multiplications in deep learning?" in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 1468–1477.
- [10] A. S. Rakin, Z. He, and D. Fan, "Bit-flip attack: Crushing neural network with progressive bit search," in *Proceedings of the IEEE International Conference on Computer Vision*, 2019, pp. 1211–1220.
- [11] —, "Tbt: Targeted neural network attack with bit trojan," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 13 198–13 207.
- [12] A. S. Rakin, M. H. I. Chowdhury, F. Yao, and D. Fan, "Deepsteal: Advanced model extractions leveraging efficient weight stealing in memories," in *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2022, pp. 1157–1174.
- [13] S. Hong, P. Frigo, Y. Kaya, C. Giuffrida, and T. Dumitras, "Terminal brain damage: Exposing the graceless degradation in deep neural networks under hardware fault attacks," in *28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 497–514.
- [14] F. Yao, A. Rakin, and D. Fan, "Deephammer: Depleting the intelligence of deep neural networksthrough targeted chain of bit flips," in *29th USENIX Security Symposium (USENIX Security 20)*, 2020.
- [15] J. Dong, H. Qiu, Y. Li, T. Zhang, Y. Li, Z. Lai, C. Zhang, and S.-T. Xia, "One-bit flip is all you need: When bit-flip attack meets model training," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 4688–4698.
- [16] J. Bai, B. Wu, Z. Li, and S.-T. Xia, "Versatile weight attack via flipping limited bits," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 11, pp. 13 653–13 665, 2023.
- [17] X. Zhou, S. Kundu, and P. A. Beerel, "What makes vision transformers robust towards bit-flip attack?" in *ICLR 2024 Workshop on Mathematical and Empirical Understanding of Foundation Models*, 2024. [Online]. Available: <https://openreview.net/forum?id=dRPvkwLxKR>
- [18] H. Chen, C. Fu, J. Zhao, and F. Koushanfar, "Proflip: Targeted trojan attack with progressive bit flips," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 7718–7727.
- [19] Y. Kim, R. Daly, J. Kim, C. Fallin, J. H. Lee, D. Lee, C. Wilkerson, K. Lai, and O. Mutlu, "Flipping bits in memory without accessing them: An experimental study of dram disturbance errors," in *ACM SIGARCH Computer Architecture News*, vol. 42, no. 3. IEEE Press, 2014, pp. 361–372.
- [20] Y. Zhang, B. Sun, W. Jiang, Y. Ha, M. Hu, and W. Zhao, "Wsq-addernet: Efficient weight standardization based quantized addernet fpga accelerator design with high-density int8 dsp-lut co-packing optimization," in *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*, 2022, pp. 1–9.
- [21] Y. Zhang, O. A. Kailani, and W. Zhao, "Addernet 2.0: Optimal fpga acceleration of addernet with activation-oriented quantization and fused bias removal based memory optimization," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.
- [22] A. S. Rakin, Z. He, J. Li, F. Yao, C. Chakrabarti, and D. Fan, "T-bfa: Targeted bit-flip adversarial weight attack," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 11, pp. 7928–7939, 2021.
- [23] M. Yan, C. W. Fletcher, and J. Torrellas, "Cache telepathy: Leveraging shared resource attacks to learn dnn architectures," in *29th USENIX Security Symposium (USENIX Security 20)*, 2020, pp. 2003–2020.
- [24] H. Luo, A. Olgun, A. G. Yağlıkçı, Y. C. Tuğrul, S. Rhyner, M. B. Cavlak, J. Lindegger, M. Sadrosadati, and O. Mutlu, "Rowpress: Amplifying read disturbance in modern dram chips," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–18.
- [25] A. Krizhevsky, V. Nair, and G. Hinton, "Cifar-10 (canadian institute for advanced research)," 2010.
- [26] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE conference on computer vision and pattern recognition*. Ieee, 2009, pp. 248–255.
- [27] I. Loshchilov and F. Hutter, "Sgdr: Stochastic gradient descent with warm restarts," *arXiv preprint arXiv:1608.03983*, 2016.
- [28] A. S. Rakin, L. Yang, J. Li, F. Yao, C. Chakrabarti, Y. Cao, J.-s. Seo, and D. Fan, "Ra-bnn: Constructing robust & accurate binary neural network to simultaneously defend adversarial bit-flip attack and improve accuracy," *arXiv preprint arXiv:2103.13813*, 2021.
- [29] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," in *International Conference on Learning Representations*, 2018. [Online]. Available: <https://openreview.net/forum?id=rJzIBfZAb>
- [30] J. Wang, Z. Zhang, M. Wang, H. Qiu, T. Zhang, Q. Li, Z. Li, T. Wei, and C. Zhang, "Aegis: Mitigating targeted bit-flip attacks against deep neural networks," *arXiv preprint arXiv:2302.13520*, 2023.
- [31] Q. Liu, J. Yin, W. Wen, C. Yang, and S. Sha, "{NeuroPots}: realtime proactive defense against {Bit-Flip} attacks in neural networks," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 6347–6364.
- [32] N. Nazari, H. M. Makrani, C. Fang, H. Sayadi, S. Rafatirad, K. N. Khasawneh, and H. Homayoun, "Forget and rewire: Enhancing the resilience of transformer-based models against {Bit-Flip} attacks," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 1349–1366.
- [33] M. Dong, Y. Wang, X. Chen, and C. Xu, "Towards stable and robust addernets," in *Advances in Neural Information Processing Systems*, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, Eds., 2021. [Online]. Available: [https://openreview.net/forum?id=hPgY4\\_gpbmU](https://openreview.net/forum?id=hPgY4_gpbmU)